

Ejercicios Resueltos

Introducción

Esta sección contiene ejercicios prácticos resueltos que integran todos los conceptos aprendidos: manipulación del DOM, eventos, estilos y JavaScript básico. Los ejercicios están ordenados de menor a mayor dificultad.

Ejercicios básicos

Ejercicio 1: Variables y tipos de datos primitivos

Declara las siguientes variables y muéstralas por consola junto al tipo de dato con el que se corresponde:

- `nombre`, con el valor `"Juan"`
- `edad`, con el valor `25`
- `altura`, con el valor `1.75`
- `esEstudiante`, con el valor `true`
- `ciudad`, sin asignarle ningún valor
- `pais`, con el valor `null`

Además, realiza las siguientes operaciones con las variables numéricas y muestra los resultados:

- Suma `5` a la edad
- Resta `3` a la edad
- Multiplica la altura por `2`
- Divide la edad entre `5`

► Solución

Conceptos practicados:

- Declaración de variables con `let`
 - Tipos de datos primitivos: `string`, `number`, `boolean`, `undefined`, `null`
 - Operador `typeof`
 - Operaciones aritméticas básicas
-

Ejercicio 2: Strings y sus métodos

Dada la siguiente cadena de texto: `"Hola Mundo desde JavaScript"`, realiza las siguientes operaciones y muestra los resultados por consola:

1. Mostrar la longitud de la cadena
2. Convertir la cadena a mayúsculas
3. Convertir la cadena a minúsculas
4. Comprobar si incluye la palabra `"Mundo"`
5. Buscar el índice donde empieza `"JavaScript"`
6. Extraer los primeros 4 caracteres usando `substring()`
7. Extraer los caracteres desde la posición 5 hasta la 10 usando `slice()`
8. Reemplazar `"Mundo"` por `"Planeta"`
9. Dividir la cadena en un array de palabras
10. Eliminar los espacios en blanco del inicio y final de `" JavaScript "`
11. Crear un mensaje usando template literals con las variables `nombre = "Ana"` y `edad = 28`

► Solución

Conceptos practicados:

- Métodos de string: `length`, `toUpperCase()`, `toLowerCase()`, `includes()`, `indexOf()`
- `substring()`, `slice()`, `replace()`, `split()`, `trim()`
- Template literals (backticks)

Ejercicio 3: Arrays - Creación y métodos básicos

Crea un array llamado `frutas` con los siguientes elementos: `"manzana"`, `"pera"`, `"naranja"`, `"plátano"`. Realiza las siguientes operaciones:

1. Mostrar por consola el array completo
2. Acceder y mostrar el primer elemento
3. Acceder y mostrar el último elemento
4. Añadir `"uva"` al final del array usando `push()`
5. Añadir `"fresa"` al principio del array usando `unshift()`
6. Eliminar el último elemento usando `pop()` y mostrar el elemento eliminado
7. Eliminar el primer elemento usando `shift()` y mostrar el elemento eliminado
8. Buscar el índice de `"pera"` usando `indexOf()`
9. Comprobar si el array incluye `"manzana"` usando `includes()`
10. Extraer los elementos desde el índice 1 hasta el 3 (sin incluir) usando `slice()`
11. Unir todos los elementos en una cadena separada por comas usando `join()`
12. Crear un array `numerosDesordenados` con los valores `[5, 2, 8, 1, 9]` y ordenarlo de menor a mayor
13. Invertir el orden del array de frutas usando `reverse()`

► Solución

Conceptos practicados:

- Creación de arrays
- Acceso a elementos por índice
- Métodos: `push()`, `pop()`, `shift()`, `unshift()`
- Métodos: `indexOf()`, `includes()`, `slice()`, `join()`
- Métodos: `sort()`, `reverse()`

Ejercicio 4: Objetos literales

Crea un objeto llamado `persona` con las siguientes propiedades:

- `nombre`: "María"
- `apellido`: "García"
- `edad`: 30
- `ciudad`: "Madrid"
- `profesion`: "Desarrolladora"
- `hobbies`: ["leer", "programar", "viajar"]
- `activo`: true

Realiza las siguientes operaciones:

1. Acceder y mostrar el nombre usando notación punto
2. Acceder y mostrar el apellido usando notación de corchetes
3. Modificar la edad a 31
4. Modificar la ciudad a "Barcelona"
5. Añadir una nueva propiedad `email` con el valor "maria@ejemplo.com"
6. Añadir una nueva propiedad `telefono` con el valor "123456789"
7. Eliminar la propiedad `activo` usando `delete`

Además, crea un objeto `calculadora` con los siguientes métodos:

- `sumar(a, b)`: retorna la suma de a y b
- `restar(a, b)`: retorna la resta de a y b
- `multiplicar(a, b)`: retorna la multiplicación de a y b
- `dividir(a, b)`: retorna la división de a entre b (controla el error de división por cero)

Prueba los métodos de la calculadora con diferentes valores.

Por último, crea un objeto `coche` con propiedades `marca`, `modelo` y `año`, y un método `descripcion()` que retorne una cadena descriptiva usando `this`.

► Solución

Conceptos practicados:

- Creación de objetos literales
- Acceso a propiedades (notación punto y corchetes)
- Modificación y adición de propiedades
- Eliminación de propiedades con `delete`
- Métodos en objetos
- Uso de `this`
- `Object.keys()`, `Object.values()`, `Object.entries()`

Ejercicio 5: Arrays de objetos

Enunciado

Crea un array llamado `estudiantes` con los siguientes objetos:

```
[  
  { id: 1, nombre: "Ana", edad: 20, nota: 8.5 },  
  { id: 2, nombre: "Carlos", edad: 22, nota: 7.0 },  
  { id: 3, nombre: "Elena", edad: 21, nota: 9.2 },  
  { id: 4, nombre: "David", edad: 23, nota: 6.8 },  
  { id: 5, nombre: "Lucía", edad: 20, nota: 8.9 }  
]
```

Realiza las siguientes operaciones:

1. Mostrar el array completo
2. Acceder y mostrar el primer estudiante
3. Acceder y mostrar el nombre del segundo estudiante
4. Añadir un nuevo estudiante: `{ id: 6, nombre: "Miguel", edad: 22, nota: 7.5 }`
5. Crear una función `buscarEstudiante(nombre)` que busque y retorne el estudiante con ese nombre
6. Filtrar y mostrar los estudiantes con nota mayor o igual a 8

7. Calcular y mostrar la nota media de todos los estudiantes
8. Modificar la nota de "Carlos" a 7.5

► Solución

Conceptos practicados:

- Arrays de objetos
- Acceso a propiedades de objetos dentro de arrays
- Iteración con `for...of`
- Búsqueda en arrays de objetos
- Filtrado de datos
- Cálculos con datos de arrays

Ejercicio 6: Estructuras de control

Realiza los siguientes ejercicios utilizando diferentes estructuras de control:

Parte 1: Condicionales

1. Crea una variable `edad` con valor `18` y usa un `if-else` para mostrar si es mayor o menor de edad
2. Crea una variable `nota` con valor `7.5` y usa `if-else if-else` para mostrar la calificación:
 - Si `nota >= 9`: "Sobresaliente"
 - Si `nota >= 7`: "Notable"
 - Si `nota >= 5`: "Aprobado"
 - Si no: "Suspenso"
3. Crea una variable `día` con valor `3` y usa `switch` para mostrar el nombre del día de la semana
4. Usa el operador ternario para crear una variable que indique si es mayor de edad

Parte 2: Bucles

5. Usa un bucle `for` para mostrar los números del 1 al 5
6. Crea un array `colores` con `["rojo", "verde", "azul", "amarillo"]` y recorre con `for` mostrando el índice y el valor
7. Recorre el array de colores usando `for...of`
8. Usa un bucle `while` para hacer una cuenta atrás desde 5 hasta 1 y mostrar "¡Despegue!"
9. Usa un bucle `do-while` para mostrar los números pares del 0 al 10
10. Usa `break` y `continue` en un bucle que muestre los números del 1 al 10, saltando el 5 y deteniendo en el 9

► Solución

Conceptos practicados:

- Condicionales: if, else if, else
- Switch-case
- Operador ternario
- Bucles: for, for...of, while, do-while
- Break y continue

Ejercicio 7: Funciones

Crea las siguientes funciones y puébalas con diferentes valores:

1. **Función declarada** `saludar(nombre)` que retorne un saludo con el nombre
2. **Función declarada** `sumar(a, b)` que retorne la suma de dos números
3. **Función sin return** `mostrarMensaje(mensaje)` que muestre un mensaje por consola
4. **Función con parámetro por defecto** `saludarConIdioma(nombre, idioma = "español")` que salude en diferentes idiomas
5. **Expresión de función** asignada a la variable `multiplicar` que multiplique dos números

6. **Arrow function** asignada a la variable `dividir` que divida dos números (controlar división por cero)
7. **Arrow function simplificada** `cuadrado` que calcule el cuadrado de un número
8. **Arrow function simplificada** `esPar` que determine si un número es par
9. **Función** `crearPersona(nombre, edad)` que retorne un objeto con esas propiedades y un método `saludar()`
10. **Función con rest parameters** `sumarTodos(...numeros)` que sume cualquier cantidad de números
11. **Función recursiva** `factorial(n)` que calcule el factorial de un número

► Solución

Conceptos practicados:

- Declaración de funciones
- Parámetros y valores de retorno
- Parámetros por defecto
- Expresiones de función
- Arrow functions
- Rest parameters (...)
- Funciones recursivas

Ejercicio 8: Métodos de arrays avanzados

Enunciado

Dado el array `numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, realiza las siguientes operaciones:

1. Usa `map()` para crear un array con los cuadrados de cada número
2. Usa `map()` para crear un array con el doble de cada número
3. Usa `filter()` para obtener solo los números pares
4. Usa `filter()` para obtener solo los números mayores que 5

5. Usa `find()` para encontrar el primer número par
6. Usa `findIndex()` para encontrar el índice del primer número par
7. Usa `some()` para verificar si hay algún número mayor que 5
8. Usa `every()` para verificar si todos los números son mayores que 0
9. Usa `reduce()` para calcular la suma de todos los números
10. Usa `reduce()` para calcular el producto de todos los números

Además, dado el siguiente array de productos:

```
let productos = [  
  { nombre: "Laptop", precio: 800, categoria: "Electrónica" },  
  { nombre: "Mouse", precio: 20, categoria: "Electrónica" },  
  { nombre: "Teclado", precio: 50, categoria: "Electrónica" },  
  { nombre: "Silla", precio: 150, categoria: "Muebles" },  
  { nombre: "Mesa", precio: 300, categoria: "Muebles" }  
];
```



Realiza:

11. Obtener un array solo con los nombres de los productos
12. Filtrar productos de la categoría "Electrónica"
13. Filtrar productos con precio menor a 100
14. Calcular el precio total de todos los productos
15. Encadenar métodos para calcular el total de la categoría "Electrónica"
16. Usar `forEach()` para mostrar todos los productos con formato

► Solución

Conceptos practicados:

- `map()`: transformar arrays
- `filter()`: filtrar elementos
- `find()` y `findIndex()`: búsqueda
- `some()` y `every()`: validaciones
- `reduce()`: reducción a un valor

- `forEach()`: iteración
 - Encadenamiento de métodos
-

Ejercicios intermedios

Ejercicio 1: Contador interactivo

Crea una página web con un contador que muestre un número y tres botones:

- Un botón "Incrementar" que aumente el contador en 1
- Un botón "Decrementar" que disminuya el contador en 1
- Un botón "Resetear" que vuelva el contador a 0

Estructura HTML requerida:

- Un `<div>` con id `contador` que muestre el número actual
- Tres botones con los ids: `btnIncrementar`, `btnDecrementar`, `btnResetear`

Requisitos adicionales:

- Cuando el contador sea positivo, el texto debe ser verde
- Cuando sea negativo, debe ser rojo
- Cuando sea cero, debe ser negro
- Usa `getElementById()` para acceder a los elementos
- Usa `addEventListener()` para manejar los clics

▶ Solución propuesta

Conceptos practicados:

- `getElementById()` para acceder a elementos
- `addEventListener()` para eventos de click
- Manipulación de `textContent`
- Manipulación de clases con `classList`

- Variables y operadores de incremento/decremento
 - Condicionales para cambiar estilos
-

Ejercicio 2: Lista de tareas simple

Crea una aplicación de lista de tareas donde el usuario pueda:

1. Escribir una tarea en un input
2. Agregar la tarea a una lista al hacer clic en un botón o presionar Enter
3. Cada tarea debe tener un botón "Eliminar" para quitarla de la lista
4. Mostrar un mensaje cuando la lista esté vacía

Estructura HTML requerida:

- Un `<input>` con id `inputTarea` para escribir la tarea
- Un botón con id `btnAgregar` para agregar la tarea
- Un `<ul>` con id `listaTareas` para mostrar las tareas

Requisitos adicionales:

- No permitir agregar tareas vacías
- Limpiar el input después de agregar una tarea
- Cada tarea debe crearse con `createElement()` y `appendChild()`

► Solución

Conceptos practicados:

- `createElement()` para crear elementos
- `appendChild()` para agregar elementos al DOM
- `remove()` para eliminar elementos
- `querySelector()` para buscar elementos
- Eventos de teclado (`keypress`)
- Validación de inputs

- Manipulación de clases y estilos
-

Ejercicio 3: Cambiar colores dinámicamente

Crea una página que permita cambiar el color de fondo de un elemento mediante botones y un input de texto:

1. Un `<div>` grande que cambiará de color
2. Tres botones con colores predefinidos (Rojo, Verde, Azul)
3. Un input donde el usuario pueda escribir cualquier color válido (nombre o código hex)
4. Un botón "Aplicar color personalizado"
5. Un botón "Color aleatorio" que genere un color random

Estructura HTML requerida:

- Un `<div>` con id `cajaColor` que será el elemento a colorear
- Botones para colores predefinidos
- Un `<input>` con id `inputColor` para colores personalizados
- Mostrar el código del color actual en un `<span>` con id `colorActual`

► Solución

Conceptos practicados:

- Manipulación de `style` para cambiar estilos inline
 - `Math.random()` y `Math.floor()` para números aleatorios
 - Conversión de números a hexadecimal con `toString(16)`
 - `padStart()` para formatear strings
 - Funciones globales y event listeners
-

Ejercicio 4: Mostrar/Ocultar contenido

Crea una página con tres secciones de contenido que se puedan mostrar u ocultar:

1. Tres botones, cada uno corresponde a una sección
2. Al hacer clic en un botón, se muestra su sección y se ocultan las demás
3. Agregar un botón "Mostrar todo" que muestre las tres secciones
4. Agregar un botón "Ocultar todo" que oculte las tres secciones

Estructura HTML requerida:

- Tres `<div>` con ids `seccion1`, `seccion2`, `seccion3`
- Botones para controlar cada sección
- Botones para mostrar/ocultar todo

Requisitos adicionales:

- Usar `classList.add()` y `classList.remove()` para manipular clases
- Crear una clase CSS `.oculto` con `display: none`
- Los botones activos deben tener un estilo diferente

► Solución

Conceptos practicados:

- `classList.add()` y `classList.remove()` para manipular clases
- Mostrar/ocultar elementos con CSS
- Switch-case para control de flujo
- Múltiples event handlers
- Gestión de estados visuales (botones activos)

Ejercicio 5: Formulario con validación

Enunciado

Crea un formulario de registro con validación en tiempo real:

Campos del formulario:

1. Nombre (mínimo 3 caracteres)
2. Email (formato válido)
3. Edad (entre 18 y 100)
4. Contraseña (mínimo 6 caracteres)

Requisitos:

- Validar cada campo cuando pierda el foco (`blur event`)
- Mostrar mensajes de error debajo de cada campo inválido
- Deshabilitar el botón de envío si hay errores
- Mostrar un resumen de los datos cuando el formulario sea válido

Estructura HTML requerida:

- Un `<form>` con id `formularioRegistro`
- Inputs con ids: `nombre`, `email`, `edad`, `password`
- `<span>` con clase `error` debajo de cada input para mensajes de error
- Un botón de envío con id `btnEnviar`
- Un `<div>` con id `resumen` para mostrar los datos

► Solución

Conceptos practicados:

- Eventos `blur` e `input` para validación
 - Expresiones regulares (regex) para validar email
 - `preventDefault()` para controlar el envío del formulario
 - Gestión de estado con objetos
 - Manipulación de atributos (`disabled`)
 - Validación de formularios en tiempo real
-

Ejercicios de Proyecto: Calculadora y Lista de Tareas

Ejercicio 1: Calculadora Básica

Crear una calculadora básica que permita realizar operaciones de suma, resta, multiplicación y división entre dos números.

A continuación se muestra el código HTML y CSS base. Deberás completar el código JavaScript para que la calculadora funcione correctamente.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Calculadora Básica</title>
  <style>
    .calculadora {
      max-width: 300px;
      margin: 50px auto;
      padding: 20px;
      border: 2px solid #333;
      border-radius: 10px;
      background-color: #f0f0f0;
    }

    .input-group {
      margin-bottom: 15px;
    }

    label {
      display: block;
      margin-bottom: 5px;
      font-weight: bold;
    }

    input[type="number"] {
      width: 100%;
      padding: 8px;
  
```

```

        border: 1px solid #ccc;
        border-radius: 4px;
        box-sizing: border-box;
    }

    .botones {
        display: grid;
        grid-template-columns: repeat(4, 1fr);
        gap: 10px;
        margin: 20px 0;
    }

    button {
        padding: 15px;
        font-size: 18px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        background-color: #007bff;
        color: white;
    }

    button:hover {
        background-color: #0056b3;
    }

    #resultado {
        font-size: 24px;
        font-weight: bold;
        text-align: center;
        padding: 15px;
        background-color: white;
        border: 2px solid #333;
        border-radius: 5px;
        margin-top: 15px;
    }

    .error {
        color: red;
    }
}
</style>
</head>
<body>
    <div class="calculadora">

```

```

<h2>Calculadora Básica</h2>

<div class="input-group">
  <label for="numero1">Primer número:</label>
  <input type="number" id="numero1" step="any">
</div>

<div class="input-group">
  <label for="numero2">Segundo número:</label>
  <input type="number" id="numero2" step="any">
</div>

<div class="botones">
  <button onclick="calcular('+')">+</button>
  <button onclick="calcular('-')">-</button>
  <button onclick="calcular('*')">*</button>
  <button onclick="calcular('/')">/</button>
</div>

<button onclick="limpiar()" style="width: 100%; background-col

  <div id="resultado">Resultado: 0</div>
</div>

<script>
  // Aquí va el código JavaScript para la calculadora
</script>
</body>
</html>

```

► Solución JavaScript

Ejercicio 2: Lista de Tareas Interactiva

Crear una aplicación de lista de tareas que permita agregar, marcar como completadas y eliminar tareas. Las tareas completadas deben mostrarse con un estilo diferente.

► Solución

Ejercicio 3: Galería de Imágenes con Lightbox

Crear una galería de imágenes con efecto lightbox. Al hacer clic en una miniatura, debe abrirse la imagen en tamaño completo con navegación entre imágenes.

► Solución

Conceptos practicados:

- Generación dinámica de elementos con `createElement()`
- Delegación de eventos
- Navegación con índices y operador módulo
- Event listeners para teclado
- Manipulación de clases para mostrar/ocultar elementos
- Deshabilitar scroll del body
- Array de objetos con datos
- Template strings para HTML dinámico